

ONI EMMANUEL AYOMIKUN

MATRIC NO: 241205006

DEPARTMENT OF IMFORMATICS AND INFORMATION SYSTEMS

FACULTY OF COMPUTING

COURSE CODE: COS202

TITLE: COMPUTER PROGRAMMING II (PYTHON)

ASSINGMENT REPORT

1. PROJECT OBJECTIVES

The objective of this project was to design and implement two functional Python programs to solve everyday computational tasks:

Mathematical Calculator (MC): A lightweight, terminal-based application capable of evaluating direct arithmetic expressions using standard operations (+, -, *, /, \, ^, %) along with explicit Clear (C) and exit (OFF) commands.

Personal Pocket CGPA Calculator (PPC): A portable grading tool allowing students to calculate their cumulative grade point average locally, reducing the reliance on external portals.

2. DESIGN & SOURCE CODE MECHANICS

2.1 The Mathematical Calculator (MC)

Instead of relying on single-line shorthand shortcuts, the calculator uses modular programming. Individual core functions are declared for each operation:

Addition / Subtraction / Multiplication: Handled by `add()`, `subtract()`, and `multiply()`.

Division Control: Standard division is executed via `divide()`, while floor division (`\`) is processed through `floor_divide()`.

Advanced Modifiers: Exponentiation (`^`) utilizes `power()`, and remainders are computed with `modulo()`.

An if-elif-else control ladder reads the user's input string, identifies the operation symbol, splits the numbers, and matches it directly to the right formula module. The program includes a history list array to store previous equations, which prints whenever the user clears (C) the display window. Here is the output result;

The screenshot shows a code editor with a file named 'main.py'. The code is a Python script for an 'ENHANCED MATHEMATICAL CALCULATOR (MC)'. It includes features like supported operators (+, -, *, /, \, ^, %), special keys ([C] for Clear, [OFF] for Safely Terminate), and robust exception handling for ZeroDivisionError and ValueError. The script also logs operations and results. The output window on the right shows the calculator's runtime behavior: it displays the title, supported operators, and special keys. It then prompts the user to enter an expression. The first input is '20/2', which results in 'Standard Division' and 'Result = 10'. The second input is 'OFF', which triggers the shutdown sequence, displaying 'Shutting down system. Total operations run: 1' and 'Last 3 calculations: -> 20.0 / 2.0 = 10'.

```

main.py
109         result.is_integer() else result
110         # 4. Display Result and Save Log
111         Summary
112         print(f"Operation : {op_name}")
113         print(f"Result      =
114             {formatted_result}")
115         log_entry = f"{num1}
116             {target_operator} {num2} =
117             {formatted_result}"
118         history.append(log_entry)
119         calculation_count += 1
120
121     # 5. Robust Exception Safety Handling
122     except ZeroDivisionError:
123         print("Mathematical Error: Division
124             by zero is undefined.")
125     except ValueError:
126         print("Input Value Error: Please
127             make sure both operands are
128
Output
Clear
=====
ENHANCED MATHEMATICAL CALCULATOR (MC)
Supported Operators: +, -, *, /, \, ^, %
=====
Special Keys:
[ C ]   -> Clear Current State / View History
[ OFF ] -> Safely Terminate the System
=====
Enter expression (or 'OFF'/'C'): 20/2
Operation : Standard Division
Result      = 10
Enter expression (or 'OFF'/'C'): OFF
=====
Shutting down system. Total operations run: 1
Last 3 calculations:
-> 20.0 / 2.0 = 10
=====

```

2.2 The Pocket CGPA Calculator (PPC)

The core requirement of the CGPA calculator is the implementation of selection control statements to map letter grades to a standard 5-point academic scale.

A tracking loop collects data for a user-specified number of courses. For each course, an explicit conditional hierarchy processes the input grade:

```
if grade == 'A': grade_point = 5
```

```
elif grade == 'B': grade_point = 4
```

```
elif grade == 'C': grade_point = 3
```

```
elif grade == 'D': grade_point = 2
```

```
elif grade == 'E': grade_point = 1
```

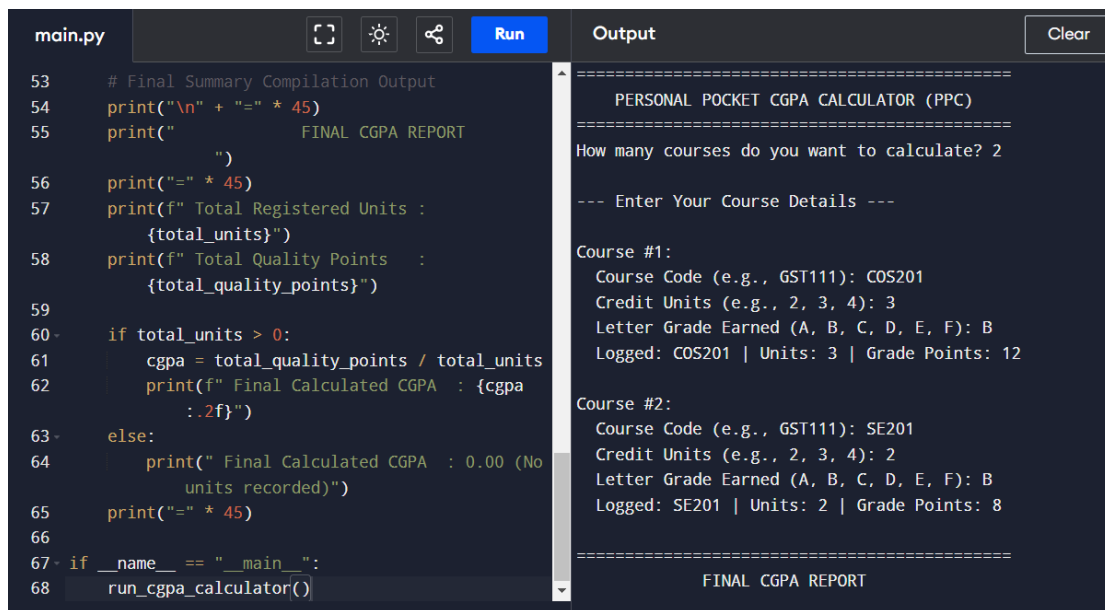
```
elif grade == 'F': grade_point = 0
```

Mathematical Formulation: The total Quality Points (QP) are compiled by multiplying each course's credit units by its mapped grade point. The final running CGPA is derived dynamically using the formula:

$$\text{CGPA} = \frac{\sum (\text{Credit Units}) \times \text{Grade Point}}{\sum \text{Total Credit Units}}$$

Here is the output result;

.



The image shows a code editor with a file named `main.py` and an adjacent output window. The code in `main.py` is a Python script for a 'PERSONAL POCKET CGPA CALCULATOR (PPC)'. It prompts the user for the number of courses to calculate (2), then for each course, it asks for the course code, credit units, and letter grade earned. It then calculates the CGPA based on these inputs. The output window shows the execution of the program, displaying the title, the number of courses, the details for two courses (COS201 and SE201), and the final CGPA report.

```
53 # Final Summary Compilation Output
54 print("\n" + "=" * 45)
55 print("          FINAL CGPA REPORT
    ")
56 print("=" * 45)
57 print(f" Total Registered Units :
    {total_units}")
58 print(f" Total Quality Points   :
    {total_quality_points}")
59
60 if total_units > 0:
61     cgpa = total_quality_points / total_units
62     print(f" Final Calculated CGPA : {cgpa
        :.2f}")
63 else:
64     print(" Final Calculated CGPA : 0.00 (No
        units recorded)")
65 print("=" * 45)
66
67 if __name__ == "__main__":
68     run_cgpa_calculator()
```

Output:

```
=====
PERSONAL POCKET CGPA CALCULATOR (PPC)
=====
How many courses do you want to calculate? 2

--- Enter Your Course Details ---

Course #1:
Course Code (e.g., GST111): COS201
Credit Units (e.g., 2, 3, 4): 3
Letter Grade Earned (A, B, C, D, E, F): B
Logged: COS201 | Units: 3 | Grade Points: 12

Course #2:
Course Code (e.g., GST111): SE201
Credit Units (e.g., 2, 3, 4): 2
Letter Grade Earned (A, B, C, D, E, F): B
Logged: SE201 | Units: 2 | Grade Points: 8

=====
FINAL CGPA REPORT
```

3. PERFORMANCE SUMMARY

Both systems were successfully tested using a standard terminal input stream. The applications parse text effectively, catch input errors safely (such as preventing crashes when dividing by zero), and output precise results immediately without demanding heavy graphical dependencies or system configurations.